

Кафедра комп'ютерних
систем і мереж

ЛАБОРАТОРНА РОБОТА №12

з дисципліни **Теорія алгоритмів і структури даних**

Тема:

Бінарні дерева

Виконала:

ст. гр. КІ-25-1

Лаврів А.

Перевірив:

Гарасимів Т.Г.

м. Івано-Франківськ

2026

Мета: Набуття практичних вмінь та навичок опрацювання нелінійних структур даних, представлених у вигляді бінарних дерев.

Завдання:

1. Написати програму, в якій дані вашого варіанту структури записуються в дерева (використати три поля для вузла – текстове дане та два числові. Наприклад, вузол дерева містить таку корисну інформацію: прізвище студента, рік народження, оцінка). Ввести з клавіатури декілька "студентів" у двійкове дерево, організоване за порядком текстового поля. Роздрукувати отримане дерево.
2. Знайти середню значення одного з числових полів, зчитуючи дані з дерева.
3. Дописати функцію видалення з пам'яті всього дерева Вказівка: рекурсивна функція, яка
 - 1) видаляє ліве піддерево, і ліву гілку занулює;
 - 2) видаляє праве піддерево, і праву гілку занулює;
 - 3) видаляє сам вузол, потім зануливши вказівник на нього Наприкінці програми видалити з пам'яті дерево.
3. "Пересипати" дані з першого дерева у друге дерево того ж типу, тільки організованого за першим числовим ключем (напр., роком народження) та роздрукувати його (а перше дерево стерти).

```
Код: #include <iostream>
#include <string>
#include <iomanip>
using namespace std;
```

```
struct Film {
    int code;
    string title;
    int year;
    string studio;

    void print() const {
        cout << "[код:" << setw(4) << code
            << " | " << left << setw(22) << title << right
            << " | рік:" << setw(5) << year
            << " | " << studio << "]\n";
    }
};
```

```

struct Node {
    Film data;
    Node* left;
    Node* right;
};

```

```

class TreeByTitle {
    Node* root;

```

```

    Node* insert(Node* node, const Film& f) {
        if (!node) {
            return new Node{f, nullptr, nullptr};
        }
        if (f.title < node->data.title)
            node->left = insert(node->left, f);
        else if (f.title > node->data.title)
            node->right = insert(node->right, f);
        else
            cout << "  [!] ФІЛЬМ \"" << f.title << "\" вже є в дереві." << endl;
        return node;
    }

```

```

void inorder(Node* node) const {
    if (!node) return;
    inorder(node->left);
    cout << " "; node->data.print(); cout << endl;
    inorder(node->right);
}

```

```

void preorder(Node* node, int depth = 0) const {
    if (!node) return;
    cout << " " << string(depth * 3, ' ');
    node->data.print();
    cout << endl;

```

```
preorder(node->left, depth + 1);  
preorder(node->right, depth + 1);  
}
```

```
void postorder(Node* node) const {  
    if (!node) return;  
    postorder(node->left);  
    postorder(node->right);  
    cout << " "; node->data.print(); cout << endl;  
}
```

```
void sumAndCount(Node* node, long long& sum, int& cnt) const {  
    if (!node) return;  
    sum += node->data.year;  
    cnt++;  
    sumAndCount(node->left, sum, cnt);  
    sumAndCount(node->right, sum, cnt);  
}
```

```
void destroy(Node*& node) {  
    if (!node) return;  
    destroy(node->left);  
    destroy(node->right);  
    delete node;  
    node = nullptr;  
}
```

```
int height(Node* node) const {  
    if (!node) return 0;  
    int lh = height(node->left);  
    int rh = height(node->right);  
    return 1 + (lh > rh ? lh : rh);  
}
```

```
int countNodes(Node* node) const {  
    if (!node) return 0;
```

```

    return 1 + countNodes(node->left) + countNodes(node->right);
}

```

```

template<typename TreeType>
void pour(Node* node, TreeType& target) const {
    if (!node) return;
    pour(node->left, target);
    target.insert(node->data);
    pour(node->right, target);
}

```

```

Node* search(Node* node, const string& title) const {
    if (!node) return nullptr;
    if (title == node->data.title) return node;
    if (title < node->data.title) return search(node->left, title);
    return search(node->right, title);
}

```

public:

```

TreeByTitle() : root(nullptr) {}

```

```

~TreeByTitle() { destroy(root); }

```

```

bool isEmpty() const { return root == nullptr; }

```

```

void insert(const Film& f) { root = insert(root, f); }

```

```

void printInorder(const string& title = "Дерево in-order (за назвою)") const {
    cout << "=== " << title
        << " | вузлів: " << countNodes(root)
        << " | висота: " << height(root) << " ===" << endl;
    if (isEmpty()) { cout << " (порожнє)" << endl; return; }
    inorder(root);
}

```

```

void printPreorder(const string& title = "Дерево pre-order") const {
    cout << "=== " << title << " ===" << endl;
}

```

```

    if (isEmpty()) { cout << " (порожнє)" << endl; return; }
    preorder(root);
}

```

```

void printPostorder(const string& title = "Дерево post-order") const {
    cout << "=== " << title << " ===" << endl;
    if (isEmpty()) { cout << " (порожнє)" << endl; return; }
    postorder(root);
}

```

```

double avgYear() const {
    if (isEmpty()) return 0.0;
    long long sum = 0; int cnt = 0;
    sumAndCount(root, sum, cnt);
    return (double)sum / cnt;
}

```

```

void destroyAll() {
    cout << " Видаляємо дерево (post-order)..." << endl;
    destroy(root);
    cout << " Дерево видалено з пам'яті." << endl;
}

```

```

void searchByTitle(const string& title) const {
    Node* found = search(root, title);
    if (found) {
        cout << " Знайдено: "; found->data.print(); cout << endl;
    } else {
        cout << " \'" << title << "\" — не знайдено." << endl;
    }
}

```

```

template<typename T>
void pourTo(T& target) const { pour(root, target); }
};

```

```

class TreeByYear {
    Node* root;

    Node* insert(Node* node, const Film& f) {
        if (!node) return new Node{f, nullptr, nullptr};
        if (f.year < node->data.year)
            node->left = insert(node->left, f);
        else
            node->right = insert(node->right, f);
        return node;
    }

    void inorder(Node* node) const {
        if (!node) return;
        inorder(node->left);
        cout << " "; node->data.print(); cout << endl;
        inorder(node->right);
    }

    void preorder(Node* node, int depth = 0) const {
        if (!node) return;
        cout << " " << string(depth * 3, ' ');
        node->data.print();
        cout << endl;
        preorder(node->left, depth + 1);
        preorder(node->right, depth + 1);
    }

    void destroy(Node*& node) {
        if (!node) return;
        destroy(node->left);
        destroy(node->right);
        delete node;
        node = nullptr;
    }

    int height(Node* node) const {
        if (!node) return 0;
        int lh = height(node->left);

```

```

    int rh = height(node->right);
    return 1 + (lh > rh ? lh : rh);
}

int countNodes(Node* node) const {
    if (!node) return 0;
    return 1 + countNodes(node->left) + countNodes(node->right);
}

public:
    TreeByYear() : root(nullptr) {}

    ~TreeByYear() { destroy(root); }

    bool isEmpty() const { return root == nullptr; }

    void insert(const Film& f) { root = insert(root, f); }

    void printInorder(const string& title = "Дерево in-order (за роком)") const {
        cout << "=== " << title
            << " | вузлів: " << countNodes(root)
            << " | висота: " << height(root) << " ===" << endl;
        if (isEmpty()) { cout << " (порожнє)" << endl; return; }
        inorder(root);
    }

    void printPreorder(const string& title = "Дерево pre-order (за роком)") const {
        cout << "=== " << title << " ===" << endl;
        if (isEmpty()) { cout << " (порожнє)" << endl; return; }
        preorder(root);
    }

    void destroyAll() {
        cout << " Видаляємо дерево за роком..." << endl;
        destroy(root);
        cout << " Дерево видалено." << endl;
    }
};

```



```
int main() {
```

```
    Film films[8] = {  
        {101, "Матриця",      1999, "Warner Bros."},  
        {205, "Інтерстеллар", 2014, "Paramount"},  
        {312, "Список Шиндлера", 1993, "Universal"},  
        {418, "Темний лицар",  2008, "Warner Bros."},  
        {524, "Форрест Гамп",   1994, "Paramount"},  
        {633, "Початок",       2010, "Warner Bros."},  
        {740, "Гладіатор",     2000, "DreamWorks"},  
        {856, "Касабланка",    1942, "Warner Bros."}  
    };
```

```
    cout << " ЗАВДАННЯ 1: Бінарне дерево за НАЗВОЮ" << endl;
```

```
    TreeByTitle tree1;
```

```
    cout << "\n--- Вводимо фільми у дерево (ключ — назва) ---" << endl;  
    for (auto& f : films) {  
        cout << " insert: \"\" << f.title << "\" (рік \" << f.year << \")\" << endl;  
        tree1.insert(f);  
    }
```

```
    cout << endl;  
    tree1.printInorder("Дерево in-order (алфавітний порядок за назвою)");
```

```
    cout << endl;  
    tree1.printPreorder("Дерево pre-order (структура дерева, відступ = рівень)");
```

```
    cout << endl;  
    tree1.printPostorder("Дерево post-order");
```

```
cout << "\n--- Спроба вставити дублікат ---" << endl;
tree1.insert({999, "Матриця", 1999, "Warner Bros."});
```

```
cout << " ЗАВДАННЯ 2: Середній рік виходу" << endl;
```

```
double avg = tree1.avgYear();
cout << "\n Середній рік виходу фільмів у дереві: "
    << fixed << setprecision(2) << avg << endl;
```

```
cout << " ПОШУК ЗА НАЗВОЮ" << endl;
```

```
cout << "\n Шукаємо \"Гладіатор\":" << endl;
tree1.searchByTitle("Гладіатор");
```

```
cout << "\n Шукаємо \"Аватар\" (відсутній):" << endl;
tree1.searchByTitle("Аватар");
```

```
cout << " ЗАВДАННЯ 3: Пересипати у дерево за РОКОМ" << endl;
```

```
TreeByYear tree2;
```

```
cout << "\n--- Пересипаємо дані з дерева-1 (назва) у дерево-2 (рік) ---" << endl;
tree1.pourTo(tree2);
cout << " Пересипання завершено." << endl;
```

```
cout << endl;
tree2.printInorder("Дерево-2 in-order (хронологічний порядок за роком)");
```

```
cout << endl;
tree2.printPreorder("Дерево-2 pre-order (структура)");
```

```
cout << endl;
tree1.printInorder("Дерево-1 (ще існує, не видалено)");
```

```
cout << " ЗАВДАННЯ 4: Видалення дерев з пам'яті" << endl;
```

```
cout << "\n--- Видаляємо дерево-1 (за назвою) ---" << endl;
tree1.destroyAll();
tree1.printInorder("Дерево-1 після destroyAll");
```

```
cout << "\n--- Видаляємо дерево-2 (за роком) ---" << endl;
tree2.destroyAll();
tree2.printInorder("Дерево-2 після destroyAll");
```

```
cout << " ДОДАТКОВО: вплив порядку вставки" << endl;
cout << " на форму дерева (за роком виходу)" << endl;
```

```
TreeByYear treeLinear;
```

```
Film sorted[5] = {
    {1, "Фільм 1942", 1942, "-"},
    {2, "Фільм 1993", 1993, "-"},
    {3, "Фільм 1999", 1999, "-"},
    {4, "Фільм 2008", 2008, "-"},
    {5, "Фільм 2021", 2021, "-"}
};
cout << "\nВставка у відсортованому порядку (рік зростає)" << endl;
for (auto& f : sorted) {
    cout << " insert рік=" << f.year << endl;
    treeLinear.insert(f);
}
treeLinear.printPreorder("Результат (вироджене праве дерево)");
```

```
TreeByYear treeMixed;
```

```
Film mixed[5] = {
```

```

    {3, "Фільм 1999", 1999, "-"},
    {1, "Фільм 1942", 1942, "-"},
    {5, "Фільм 2021", 2021, "-"},
    {2, "Фільм 1993", 1993, "-"},
    {4, "Фільм 2008", 2008, "-"}
};

cout << "\nВставка у змішаному порядку (1999, 1942, 2021, 1993, 2008):" << endl;
for (auto& f : mixed) {
    cout << " insert рік=" << f.year << endl;
    treeMixed.insert(f);
}
treeMixed.printPreorder("Результат (збалансованіше дерево)");

cout << "\n--- Висновок ---" << endl;
cout << "In-order обхід BST завжди дає відсортовану послідовність." << endl;
cout << "Порядок вставки впливає на форму та висоту дерева." << endl;
cout << "\"Пересипання\" = обхід першого дерева + вставка у друге." << endl;

return 0;
}

```

```

--- Вводимо фільми у дерево (ключ – назва) ---
insert: "Матриця" (рік 1999)
insert: "Інтерстеллар" (рік 2014)
insert: "Список Шиндлера" (рік 1993)
insert: "Темний лицар" (рік 2008)
insert: "Форрест Гамп" (рік 1994)
insert: "Початок" (рік 2010)
insert: "Гладіатор" (рік 2000)
insert: "Касабланка" (рік 1942)

=== Дерево in-order (алфавітний порядок за назвою) | вузлів: 8 | висота: 4 ===
[код: 205 | Інтерстеллар | рік: 2014 | Paramount]
[код: 740 | Гладіатор | рік: 2000 | DreamWorks]
[код: 856 | Касабланка | рік: 1942 | Warner Bros.]
[код: 101 | Матриця | рік: 1999 | Warner Bros.]
[код: 633 | Початок | рік: 2010 | Warner Bros.]
[код: 312 | Список Шиндлера | рік: 1993 | Universal]
[код: 418 | Темний лицар | рік: 2008 | Warner Bros.]
[код: 524 | Форрест Гамп | рік: 1994 | Paramount]

=== Дерево pre-order (структура дерева, відступ = рівень) ===
[код: 101 | Матриця | рік: 1999 | Warner Bros.]
  [код: 205 | Інтерстеллар | рік: 2014 | Paramount]
    [код: 740 | Гладіатор | рік: 2000 | DreamWorks]
      [код: 856 | Касабланка | рік: 1942 | Warner Bros.]
        [код: 312 | Список Шиндлера | рік: 1993 | Universal]
          [код: 633 | Початок | рік: 2010 | Warner Bros.]
            [код: 418 | Темний лицар | рік: 2008 | Warner Bros.]
              [код: 524 | Форрест Гамп | рік: 1994 | Paramount]

```

Рис. 1 - Демонстрація роботи коду

```

=== Дерево post-order ===
[код: 856 | Касабланка | рік: 1942 | Warner Bros.]
[код: 740 | Гладіатор | рік: 2000 | DreamWorks]
[код: 205 | Інтерстеллар | рік: 2014 | Paramount]
[код: 633 | Початок | рік: 2010 | Warner Bros.]
[код: 524 | Форрест Гамп | рік: 1994 | Paramount]
[код: 418 | Темний лицар | рік: 2008 | Warner Bros.]
[код: 312 | Список Шиндлера | рік: 1993 | Universal]
[код: 101 | Матриця | рік: 1999 | Warner Bros.]

--- Спроба вставити дублікат ---
[!] Фільм "Матриця" вже є в дереві.
ЗАВДАННЯ 2: Середній рік виходу

Середній рік виходу фільмів у дереві: 1995.00
ПОШУК ЗА НАЗВОЮ

Шукаємо "Гладіатор":
Знайдено: [код: 740 | Гладіатор | рік: 2000 | DreamWorks]

Шукаємо "Аватар" (відсутній):
"Аватар" – не знайдено.
ЗАВДАННЯ 3: Пересипати у дерево за РОКОМ

--- Пересипаємо дані з дерева-1 (назва) у дерево-2 (рік) ---
Пересипання завершено.

```

Рис.2 - Демонстрація роботи коду

```

=== Дерево-2 in-order (хронологічний порядок за роком) | вузлів: 8 | висота: 6 ===
[код: 856 | Касабланка | рік: 1942 | Warner Bros.]
[код: 312 | Список Шиндлера | рік: 1993 | Universal]
[код: 524 | Форрест Гамп | рік: 1994 | Paramount]
[код: 101 | Матриця | рік: 1999 | Warner Bros.]
[код: 740 | Гладіатор | рік: 2000 | DreamWorks]
[код: 418 | Темний лицар | рік: 2008 | Warner Bros.]
[код: 633 | Початок | рік: 2010 | Warner Bros.]
[код: 205 | Інтерстеллар | рік: 2014 | Paramount]

=== Дерево-2 pre-order (структура) ===
[код: 205 | Інтерстеллар | рік: 2014 | Paramount]
  [код: 740 | Гладіатор | рік: 2000 | DreamWorks]
    [код: 856 | Касабланка | рік: 1942 | Warner Bros.]
      [код: 101 | Матриця | рік: 1999 | Warner Bros.]
        [код: 312 | Список Шиндлера | рік: 1993 | Universal]
          [код: 524 | Форрест Гамп | рік: 1994 | Paramount]
            [код: 633 | Початок | рік: 2010 | Warner Bros.]
              [код: 418 | Темний лицар | рік: 2008 | Warner Bros.]

=== Дерево-1 (ще існує, не видалено) | вузлів: 8 | висота: 4 ===
[код: 205 | Інтерстеллар | рік: 2014 | Paramount]
[код: 740 | Гладіатор | рік: 2000 | DreamWorks]
[код: 856 | Касабланка | рік: 1942 | Warner Bros.]
[код: 101 | Матриця | рік: 1999 | Warner Bros.]
[код: 633 | Початок | рік: 2010 | Warner Bros.]
[код: 312 | Список Шиндлера | рік: 1993 | Universal]
[код: 418 | Темний лицар | рік: 2008 | Warner Bros.]
[код: 524 | Форрест Гамп | рік: 1994 | Paramount]
ЗАВДАННЯ 4: Видалення дерев з пам'яті

```

Рис. 3 - Демонстрація роботи коду

```

--- Видаляємо дерево-1 (за назвою) ---
Видаляємо дерево (post-order)...
Дерево видалено з пам'яті.
=== Дерево-1 після destroyAll | вузлів: 0 | висота: 0 ===
(порожнє)

--- Видаляємо дерево-2 (за роком) ---
Видаляємо дерево за роком...
Дерево видалено.
=== Дерево-2 після destroyAll | вузлів: 0 | висота: 0 ===
(порожнє)
ДОДАТКОВО: вплив порядку вставки
на форму дерева (за роком виходу)

Вставка у відсортованому порядку (рік зростає):
insert рік=1942
insert рік=1993
insert рік=1999
insert рік=2008
insert рік=2021
=== Результат (вироджене праве дерево) ===
[код: 1 | Фільм 1942 | рік: 1942 | -]
  [код: 2 | Фільм 1993 | рік: 1993 | -]
    [код: 3 | Фільм 1999 | рік: 1999 | -]
      [код: 4 | Фільм 2008 | рік: 2008 | -]
        [код: 5 | Фільм 2021 | рік: 2021 | -]

```

Рис. 4 - Демонстрація роботи коду

```

Вставка у змішаному порядку (1999, 1942, 2021, 1993, 2008):
insert рік=1999
insert рік=1942
insert рік=2021
insert рік=1993
insert рік=2008
=== Результат (збалансованіше дерево) ===
[код: 3 | Фільм 1999 | рік: 1999 | -]
  [код: 1 | Фільм 1942 | рік: 1942 | -]
    [код: 2 | Фільм 1993 | рік: 1993 | -]
      [код: 5 | Фільм 2021 | рік: 2021 | -]
        [код: 4 | Фільм 2008 | рік: 2008 | -]

--- Висновок ---
In-order обхід BST завжди дає відсортовану послідовність.
Порядок вставки впливає на форму та висоту дерева.
"Пересипання" = обхід першого дерева + вставка у друге.

```

Рис. 4 - Демонстрація роботи коду

Висновок: У ході виконання лабораторної роботи було досліджено принципи побудови та роботи з бінарними деревами. Було реалізовано основні операції роботи з деревами: додавання вузлів, перегляд дерева та видалення елементів. Отримано практичні навички використання рекурсивних алгоритмів та роботи з нелінійними структурами даних мовою C++.